DEAD LOCK SITUATION :

01 March 2025 09:21

When does a Deadlock Situation Occur?

- Deadlock occurs when **two or more threads are waiting for each other to release resources**, and none of them can proceed because they are stuck waiting. This is called a **Deadlock**.

Real-Life Example for Students

Imagine there are two students — **Rahul** and **Priya**.

- Rahul has a **Pen** but he needs a **Notebook**.
- Priya has a **Notebook** but she needs a **Pen**.

Now both of them are waiting for each other:

- Rahul says — *"Give me the notebook first, then I will write."*
- Priya says — *"Give me the pen first, then I will write."*

Result? — Both are stuck, and neither of them can proceed. This is Deadlock.

```
package synchronization;

class Pen {
    synchronized void usePen(Notebook notebook) {
        System.out.println(Thread.currentThread().getName() + " locked Pen, wants Notebook");
        synchronized (notebook) {
            System.out.println(Thread.currentThread().getName() + " got both Pen and Notebook");
        }
    }
}

class Notebook {
    synchronized void useNotebook(Pen pen) {
        System.out.println(Thread.currentThread().getName() + " locked Notebook, wants Pen");
        synchronized (pen) {
            System.out.println(Thread.currentThread().getName() + " got both Notebook and Pen");
        }
    }
}

public class DeadlockDemo {
    public static void main(String[] args) {
        Pen pen = new Pen();
        Notebook notebook = new Notebook();

        Thread t1 = new Thread(() -> pen.usePen(notebook), "PRIYA");
        Thread t2 = new Thread(() -> notebook.useNotebook(pen), "RAHUL");

        t1.start();
        t2.start();
    }
}
```

[PRIYA]
Holds: Pen
Wants: Notebook (locked by RAHUL)

[RAHUL]
Holds: Notebook
Wants: Pen (locked by PRIYA)

This is called **circular wait** — the core reason for deadlock.

Solution : Lock Ordering Rule

👉 ***"Always take locks in the same order in all threads."***

Example:

- Both threads should **ALWAYS lock Pen first, then Notebook**.
- OR both should **ALWAYS lock Notebook first, then Pen**.

```
class Pen {
```

```
    void usePen(Notebook notebook) {
        synchronized (Pen.class) {  // Lock 1
            System.out.println(Thread.currentThread().getName() + " locked Pen, wants Notebook");
            synchronized (Notebook.class) {  // Lock 2
                System.out.println(Thread.currentThread().getName() + " got both Pen and Notebook");
            }
        }
    }
}

class Notebook {
    void useNotebook(Pen pen) {
        synchronized (Pen.class) {  // Lock 1 (same order as above!)
            System.out.println(Thread.currentThread().getName() + " locked Pen, wants Notebook");
            synchronized (Notebook.class) {  // Lock 2
                System.out.println(Thread.currentThread().getName() + " got both Notebook and Pen");
            }
        }
    }
}


public class DeadlockDemo {
    public static void main(String[] args) {
        Pen pen = new Pen();
        Notebook notebook = new Notebook();

        Thread t1 = new Thread(() -> pen.usePen(notebook), "PRIYA");
        Thread t2 = new Thread(() -> notebook.useNotebook(pen), "RAHUL");

        t1.start();
        t2.start();
    }
}
```

- In **both methods**, we take **Pen lock first**, then **Notebook lock**.
- This **prevents circular wait**.
- Since both threads agree on **same lock order**, no deadlock can happen!